Section A: Attack Methodology

Q1)

I started off by putting each hash through Crackstation so that I could eliminate easy hashes and not have to waste time doing them in bulk on my CPU. To use crack station, I split them up into groups of 20. That provided the passwords for 27 of the hashes leaving me with 18. I then moved on to using Hashcat dictionary attacks, going from small dictionaries with no rules to largest dictionaries with multiple, so that I could eliminate less secure hashes before wasting time on later stages. I first ran ruleless on the xato-net-10-million-passwords dictionary, but no new passwords were discovered. I then added the OneRuleToRuleThemAll.rule I found on GitHub with the same dictionary, and nine more were discovered. Having trimmed the remaining down to only 9 passwords, I could afford to use a very thorough dictionary attack. I decided on 'kaonashi' which contains over 9gb of passwords, with the OneRule from earlier.

Q2)

Crackstation was able to break the most passwords by far almost instantaneously, with 27 out of the original 45. Crackstation works by matching hashes against ones precomputed from the top couple million most common passwords. This tells me the majority of people are not creative and will use very simple passwords such as words, dates, or other numeric strings that are so common that they had been previously leaked.

Q3)

The least hashes were cracked using the basic dictionary attack with no rules, which was 9 of the remaining 18. To resist a dictionary attack, the password should:
1. Be long and random enough that no rules or transformations could reproduce it a reasonable time. Most of these dictionaries only contain passwords up to around 15 characters, beyond that brute forcing becomes unviable.

2. Not contain any words in the dictionary, even with mutations like a -> @ and l -> ! because rules can break those almost instantly.

Section B: Hash Identification

Q4)

If the second one fails on m=0 and you know that the password is in the dictionary, this means that the second hash is not an MD5 hash. I would first try different m values on the same table. The other most common 32 character hash is NTLM, which has a

hashcat mode of 1000. If that doesn't work, then try all the 32 bit hashes in hashcat –example-hashes. I would also double check the encoding on the characters in the file, it is possible that it is some kind of encoded hash in a hex string.
If none of those options work, the hash could be salted in which case you would have to explicitly provide it. A dictionary attack on a salted hash would be near impossible otherwise.

Q5)

I would start with the 32 character hashes, these are probably MD5s which are super fast to compute. I would run these through hashcat on mode 0 and extract these passwords first.

I would then move onto the SHA256 hashes, which are also very fast to compute, so I would put them through a rainbow table as well, saving as many passwords as possible.

If I still had time after that, I would try to chain rules to extract any MD5 or SHA256 hashes that I might have missed on the first passthrough.

Finally, if everything else was cracked, I would approach the hashes beginning with $2b$12. The header means they use the Bcrypt algorithm with a cost factor of 12, and that each hash is salted, and must run through the hash algorithm 2^12 times. You are unlikely to break any of these unless the password is literally one character long.

Algorithms like SHA256 and MD5 are useful in specific circumstances like signing, but hashes need to be built to circumvent specific attacks like rainbow tables and hardware speedup through GPU acceleration. Bcrypt does both by salting and slowing itself down.

Section C: Benchmarking and Speed

Q6)

Output from MD5:

```
METAL API (Metal 373.7)
=======================
* Device #01: Apple M2 Pro, skipped

OpenCL API (OpenCL 1.2 (Jul 31 2026 20:36:30)) - Platform #1 [Apple]
====================================================================
* Device #02: Apple M2 Pro, GPU, 6062/12124 MB (1136 MB allocatable), 19MCU

Benchmark relevant options:
===========================
* --backend-devices-virtmulti=1
* --backend-devices-virthost=1
* --optimized-kernel-enable
```

```
------------------
* Hash-Mode 0 (MD5)
------------------

Speed.#02.....:  3656.4 MH/s (7.18ms) @ Accel:224 Loops:1024 Thr:256 Vec:1

Started: Sun Oct 18 17:38:58 2026
Stopped: Sun Oct 18 17:39:11 2026
```

Output from SHA256

```
METAL API (Metal 373.7)
=======================
* Device #01: Apple M2 Pro, skipped

OpenCL API (OpenCL 1.2 (Jul 31 2026 20:36:30)) - Platform #1 [Apple]
====================================================================
* Device #02: Apple M2 Pro, GPU, 6062/12124 MB (1136 MB allocatable), 19MCU

Benchmark relevant options:
===========================
* --backend-devices-virtmulti=1
* --backend-devices-virthost=1
* --optimized-kernel-enable

-------------------------------------------------------------------
* Hash-Mode 3200 (bcrypt $2*$, Blowfish (Unix)) [Iterations: 32]
-------------------------------------------------------------------

Speed.#02.....:     5420 H/s (306.09ms) @ Accel:462 Loops:32 Thr:8 Vec:1

Started: Sun Oct 18 17:39:40 2026
Stopped: Sun Oct 18 17:40:55 2026
```

My computer was able to compute 3656.4 MH/s using MD5, and 5420 H/s using Bcrypt with cost factor 5 (32 iterations). Dividing gives the ratio: 696,589.83. This means that for every single Bcrypt hash calculated, you could compute almost 700,000 MD5s.

Q7)

To generate hashes for all 26^8 lowercase passwords, my computer would take:

MD5:

(26^8 / 3,656,400,000) seconds

57.11 seconds – under a minute

Bcrypt (cost factor 5):

(26^8 / 5420) seconds

39,852,493.24 seconds

39,852,493.24 / 31,536,000 years

1.26 years

Section D

Q8)

Matching password hashes between two users means that the users share the same password (or that their passwords have colliding hashes, either way the same password can be used for both). In an unsalted database, this means that an attacker would be able to speed up the cracking process if they eliminated duplicate hashes. If they know the password of one of the accounts through another attack (phishing, keylogging, etc.) they can access duplicate hash accounts without having to crack any hashes.

Q9)

Even if the attacker was able to obtain the salt for each password, a rainbow table attack still wouldn't work. This is because rainbow tables store precomputed hashes per password:

MD5(password) = password

so

MD5(password + salt)

has no matching plaintext entry in the table.

For the table to work, it would have to have every password computed with every salt in the database.

This slows down attackers greatly, because instead of a very quick look up in the existing table, the attacker must recompute all the hashes. As shown above, salted password cracking – especially with Bcrypt takes a ton of compute and a very long time.

Most hash algorithms can be computed very quickly as they are just iterations of bitwise operations. The main slowdown comes from the intentional iterations added, such as the cost factor on Bcrypt. This limits the attackers speed because it forces them to calculate thousands of hashes per password instead of just 1.

Q10)

The correct part of that claim is that it defeats rainbow tables, which rely on the strings being unsalted as previously explained. The critical vulnerability is the use of the MD5 hash function, which can be computed too fast by attackers to be secure, even with the addition of salts.

To compute a rainbow table for all lowercase strings of length 6, 26^6 in total, it would take 26^6 / 3,656,400,000 seconds, less than 80ms for MD5. And 26^6 / 5420 = 58,953 seconds for Bcrypt, more than 16 hours.

The slowdown from bcrypt is significant. At this speed the entire MD5 database could be cracked instantly, while a database of just 1000 users takes almost 2 years with bcrypt.

Section E: Explaining the Code

Q12)

I used the bcrypt library because the implementation is extremely simple.

Using bcrypt is better than MD5 because its almost 700,000 times faster to brute force an MD5 hash.

Q13)

Hashing password “Hello1234” twice yields:

1. $2b$12$WdiIV.L.Y/nZs85YRk4IOe9ZmPNz1apfFK9N.ihhdZMhskXcXDJiK
2. $2b$12$pRh6zXd.3HyHONX7n6kHx.DRbu52WG5kXLa4xkDfgRziVXqRTXT1y

These hashes are complety different, meaning the salts applied to the passwords each time are random and unique.

First, stored_hash is saved from the first time the password string is hashed. My verify_password(password, stored_hash) function calls bcrypt.checkpw internally. That calls bcrypt.hashpw(password) using the salt embedded in the first 29 characters of stored_hash. The resulting hash is compared to stored_hash. If they match, that means the passwords match too.

Q14)

1. Rainbow tables. With MD5 hashes, each password could be looked up and cracked almost immediately. In my fixed version, the random salting makes this impossible.
2. Brute force hash verification. MD5 hashes can be generated and checked against password databases extremely fast. With my implementation, the attacker must

generate 2^12 times more hashes due to the cost factor. With special characters and alphanumeric strings, even 8 character passwords would take thousands of years on my computer, making the attack unviable.

Q15)

1) Minium length: 10 characters. Exponential increase of password strength with character count made uma.lewis's '934130589' much harder to crack than passwords containing special characters and random strings that were shorter. Passwords shorter than 6 characters are too prone to repetition and many have an exact match in CrackStation: ultra, xtga, 43019. Using 10 characters makes duplication rare. Additionally, most password databases store the greatest number of 5-8 character passwords and few 10+ chars because of their rarity.
2) Charset: at least 1 upper, 1 lower, 1 digit, and 1 special character. Lowercase only words preformed the worst of any category, every single one was immediately discovered in the first crackstation passthrough. Even very short strings that had a special character or digit like "humm3l!" and "d0g!" took longer and required rules to discover. Requiring these characters forces the attackers to put computationally expensive rules on.
3) Dictionary blocker: Yes, in an ideal world this would make passwords much, much rarer as they just become random strings, invalidating rainbow tables and any kind of dictionary attack. It would block passwords like "freckles, chopper, dynamite, bright, ultra" which were all super easy to break.
4) Resistances
    a. Long minimum length counteracts every brute force method
    b. Character set requirements blocks a dictionary attack by making passwords less common
    c. Dictionary blocker stops rule based transformations of passwords that can seem complex but are breakable: "Vagabond123"
5) One inconvenience is that passwords become virtually impossible to remember. Most common passwords are dictionary words or proper nouns with swapped special characters and trailing numbers. With dictionary blocking, users have to come up with entirely new passwords and remember them. If this was a production app, I would drop the dictionary block, as the 10 character length combined with the character set is enough to ensure most passwords become very hard to break.